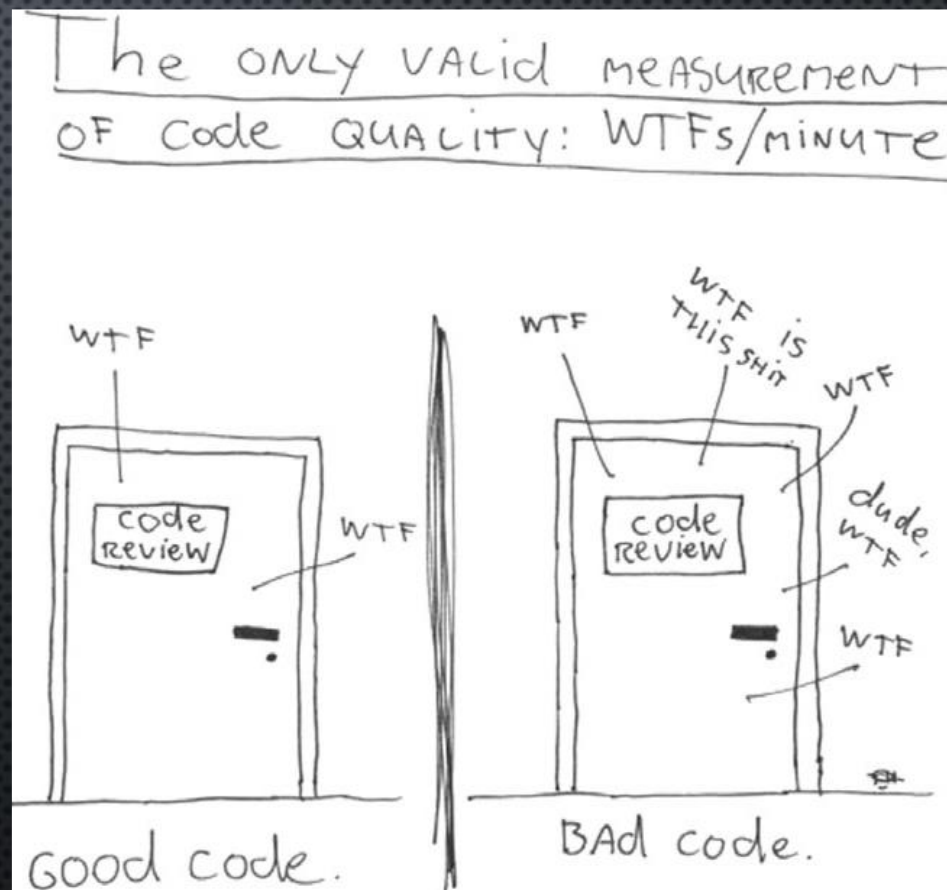


ООП 2

ПРАВИЛЕН КОД

СЪЩНОСТ

- КАЧЕСТВЕН КОД?



КАКВО Е КАЧЕСТВЕН КОД?

- ЧЕТИМ КОД
- ПРЕДВИДИМ КОД
- ЛЕСЕН ЗА ПОДДРЪЖКА
- ЛЕСЕН ЗА DEBUGGING
- ДОСТАТЪЧНО ПРОСТ
- ДОСТАТЪЧНО ГЪВКАВ

Качественият код НЕ означава много design patterns и сложна архитектура.

КАКВО Е КАЧЕСТВЕН КОД?

- ЛОШИЯТ КОД УВЕЛИЧАВА COGNITIVE LOAD (УМСТВЕНО НАТОВАРВАНЕ)
- РАЗРАБОТЧИКЪТ ТРЯБВА ДА ПОМНИ ТВЪРДЕ МНОГО
- ТРУДНО СЕ ПРОСЛЕДЯВА ЛОГИКАТА
- УВЕЛИЧАВА СЕ ШАНСЪТ ЗА ГРЕШКИ

Почти всички clean code проблеми реално са cognitive load проблеми.

ПРИМЕР ЗА COGNITIVE LOAD

- ЛОШИТЕ ИМЕНА КАРАТ РАЗРАБОТЧИКА ДА ГАДАЕ
- КОДЪТ СТАВА ТРУДЕН ЗА ЧЕТЕНЕ
- ЛИПСВА ЯСНА КОМУНИКАЦИЯ

```
int d;  
int a = 1;  
if ( 0 == 1 ){  
a=01;  
} else  
l=01;  
...
```

```
int elapsedDays;
```

Добрият код комуникира intent-а директно.

ИМЕНУВАНЕ (NAMING)

- ИМЕНАТА СА КОМУНИКАЦИЯ
- КОДЪТ СЕ ЧЕТЕ ПОВЕЧЕ, ОТКОЛКОТО СЕ ПИШЕ
- ИМЕТО ТРЯБВА ДА РАЗКРИВА INTENT-А
- ИЗБЯГВАЙТЕ ПОДВЕЖДАЩИ ИМЕНА

Naming е един от най-важните фактори за readability.

ИЗБЯГВАНЕ НА ДЕЗИНФОРМАЦИЯ

- LIST<USER> USERMAP Е ПОДВЕЖДАЩО
- USERDATA Е ТВЪРДЕ ОБЩО
- MANAGER И PROCESSOR ЧЕСТО НЕ КАЗВАТ ПОЧТИ НИЩО

```
List<User> userMap;
```

```
Map<String, User> usersById;
```

Твърде общите имена увеличават cognitive load.

ПРОИЗНОСИМИ ИМЕНА

- USRNM Е ТРУДНО ЗА ЧЕТЕНЕ
- USERNAME Е ЯСНО И ПРОИЗНОСИМО
- КОДЪТ ТРЯБВА ДА СЕ ОБСЪЖДА ЛЕСНО

Добрата комуникация е важна при code review и debugging.

ИМЕНУВАНЕ (NAMING)

- ИЗБЯГВАНЕ НА МЕНТАЛНИ ДЕФИНИЦИИ
 - ЕДНОБУКВЕНИ РЕСУРСИ
- ИМЕНА НА КЛАСОВЕ – СЪЩЕСТВИТЕЛНО
- ИМЕНА НА МЕТОДИ – ГЛАГОЛ

СТРУКТУРИРАНЕ НА КОД. МЕТОДИ

- МЕТОДЪТ НЕ ТРЯБВА ДА ИМА МНОГО RESPONSIBILITIES
- ВАЛИДИРАНЕ + DATABASE + EMAIL + PDF = ПРОБЛЕМ
- ТВЪРДЕ МНОГО ПРИЧИНИ ЗА ПРОМЯНА

```
ProcessorOrder () {  
  validate();  
  save();  
  sendEmail();  
  generatePdf();  
}
```

Single Responsibility Principle важи и за методите.

ЕДНО НИВО НА АБСТРАКЦИЯ

- НЕ СМЕСВАЙТЕ BUSINESS LOGIC С LOW-LEVEL ОПЕРАЦИИ
- НЕ СМЕСВАЙТЕ SQL И DOMAIN LOGIC
- НЕ СМЕСВАЙТЕ NETWORKING И VALIDATION

Методът трябва да работи на едно концептуално ниво.

СТРУКТУРИРАНЕ НА КОД. МЕТОДИ

- СТРЕМЕЖ КЪМ МИНИМАЛНИ ВХОДНИ ПАРАМЕТРИ
 - БУЛЕВ ПАРАМЕТЪР – МНОГО ЛОША ПРАКТИКА
- КОМБИНАЦИЯ В ОБЕКТ
- ИЗБЯГВАНЕ НА СКРИТИ ЕФЕКТИ
- ИЛИ ДА ПРАВИ НЕЩО ИЛИ ДА ВРЪЩА НЕЩО
- ИЗПОЛЗВАНЕ НА EXCEPTIONS ВМЕСТО КОДОВЕ ЗА ГРЕШКА
- TRY-CATCH В ОТДЕЛЕН МЕТОД
- DRY

DRY

- DRY = DON'T REPEAT YOURSELF
- НЕ ДУБЛИРАЙ KNOWLEDGE
- НО НЕ АБСТРАХИРАЙ ВСЯКО СХОДСТВО

Повторение в кода не винаги означава повторение на логика или смисъл

DRY

```
double calculateOrderTax(double price) {  
    return price * 0.20;  
}  
  
double calculateInvoiceTax(double price) {  
    return price * 0.20;  
}
```

```
private static final double VAT_RATE = 0.20;  
  
double calculateTax(double price) {  
    return price * VAT_RATE;  
}
```

DRY

```
double calculateStudentDiscount(double price) {  
    return price * 0.10;  
}
```

```
double calculateTeacherDiscount(double price) {  
    return price * 0.10;  
}
```

OVER-ABSTRACTION

- ПРЕКАЛЕНАТА ABSTRACTION УВЕЛИЧАВА COMPLEXITY
- ТРУДНО DEBUGGING
- ТРУДНА НАВИГАЦИЯ

GenericAbstractBaseManagerFactoryProcessor

Повече abstraction НЕ означава автоматично по-добър дизайн.

YAGNI

- YOU AREN'T GONNA NEED IT
- НЕ ДОБАВЯЙ БЪДЕЩА ГЪВКАВОСТ БЕЗ НУЖДА
- ИЗБЯГВАЙ FAKE FLEXIBILITY

В много enterprise проекти се добавят прекалено много абстракции без реална нужда.

КОМЕНТАРИ

- ИЗПОЛЗВАНЕТО НА КОМЕНТАР Е ПОРАДИ НЕВЪЗМОЖНОСТ КОДА САМ ДА ПОЯСНИ
- ЗНАЧЕЩИ КОМЕНТАРИ
 - КОМЕНТАР НА ПРОМЕНЛИВА - +/-
- ДОБРИЯТ КОМЕНТАР НЕ ОПРАВЯ ЛОШИЯ КОД
- WHY COMMENTS ЧЕСТО СА ВАЖНИ
- ДОКУМЕНТАЦИЯ - JAVADOCS

Някои неща не могат да се обяснят само с код.

COUPLING И COHESION

- HIGH COHESION = ЯСНА RESPONSIBILITY
- LOW COUPLING = КЛАСОВЕТЕ ЗНАЯТ МАЛКО ЕДИН ЗА ДРУГ
- STRONG COUPLING ВОДИ ДО FRAGILE CODE

Това са едни от най-важните архитектурни принципи.

SOLID

- SINGLE RESPONSIBILITY PRINCIPLE
- OPEN CLOSED PRINCIPLE
- LISKOV SUBSTITUTION PRINCIPLE
- INTERFACE SEGREGATION PRINCIPLE
- DEPENDENCY INVERSION PRINCIPLE

SOLID не е религия, а набор от engineering guidelines.

ОБЕКТИ И СТРУКТУРИ

- ОБЕКТИ — СКРИТИ ДАННИ , ОТКРИТО ПОВЕДЕНИЕ
- СТРУКТУРИ — ОТКРИТИ ДАННИ С МНОГО МАЛКО ПОВЕДЕНИЕ

КЛАСОВЕ

- МАЛКИ И ПРОСТИ
 - **SOLID**

CLEAN CODE МИТОВЕ

- SWITCH ВИНАГИ Е ЛОШ
- ВСИЧКО ТРЯБВА ДА Е INTERFACE
- ПОВЕЧЕ ABSTRACTION = ПО-ДОБЪР КОД
- МАЛКИТЕ МЕТОДИ ВИНАГИ СА ПО-ДОБРИ

Software engineering е баланс, а не догма.

ФИНАЛНИ ИЗВОДИ

- КАЧЕСТВЕНИЯТ КОД НАМАЛЯВА COGNITIVE LOAD
- КОМУНИКИРА INTENT
- ИЗБЯГВА НЕНУЖНА COMPLEXITY
- ПРАВИ СИСТЕМАТА MAINTAINABLE

Добрия инженеринг = правилен баланс.